

Especificación técnica del backend — Generador de planes nutricionales

Documento de arquitectura y especificación para construir el backend con Claude Code. Versión del documento: 1.0. Escrito para un proyecto que debe ser **robusto desde el día 1 y escalar horizontalmente sin re-arquitectura**.

0. Cómo usar este documento

Este archivo es el contrato de construcción. Entrégalo completo a Claude Code y constrúyelo **por módulos, en el orden de la sección 22 (Roadmap)**. Cada módulo tiene: su responsabilidad, sus contratos de entrada/salida (schemas), sus reglas de negocio y sus pruebas mínimas. No avances al siguiente módulo sin que el anterior tenga sus pruebas en verde.

Regla mental para todo el proyecto: **el código es dueño de los números; la IA es dueña del lenguaje y la selección**. Cuando dudes de quién hace algo, esa frase decide.

Nota sobre "infinitamente escalable": ningún sistema es literalmente infinito. Lo que este diseño garantiza es que **el núcleo de dominio nunca se reescribe** al crecer; solo se cambian o agregan *adaptadores* (UI, base de datos, cola de trabajos, proveedor de IA). Esa es la propiedad que quita los techos habituales.

1. Propósito y alcance

Sistema backend que, a partir de la información de un cliente (macros objetivo, lista de alimentos que le gustan, restricciones), **genera un plan nutricional de 30 días** (dos ciclos de 15 días, formato de 5 comidas: desayuno, snack AM, almuerzo, snack PM, cena) y lo exporta a PDF/DOCX con la marca del entrenador.

Entrada típica: un documento Word de *intake*. Salida: un plan revisable y exportable.

El sistema debe servir dos escenarios con el **mismo núcleo**:

1. **Nivel 1 — mono-usuario local**: una sola entrenadora (Valeria), corriendo en su máquina, interfaz Streamlit, base SQLite. Sin servidor, sin cuentas.

2. **Nivel 2+ — multi-entrenador SaaS:** muchos entrenadores, API web, base Postgres, cuentas, cobros, trabajos asíncronos.

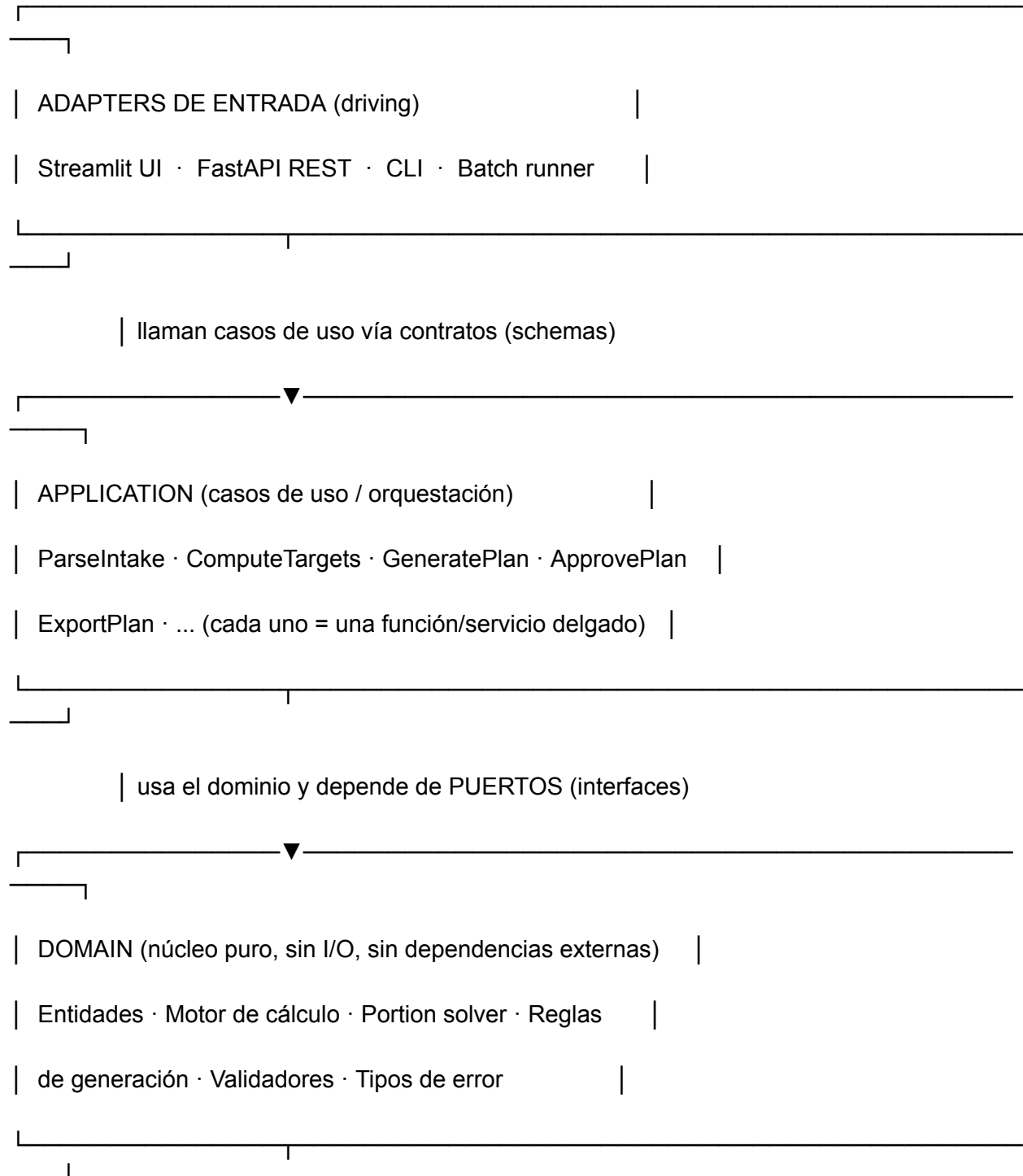
El requisito no negociable: **construir el Nivel 1 de forma que convertirse en Nivel 2+ no requiera tocar la lógica de negocio.**

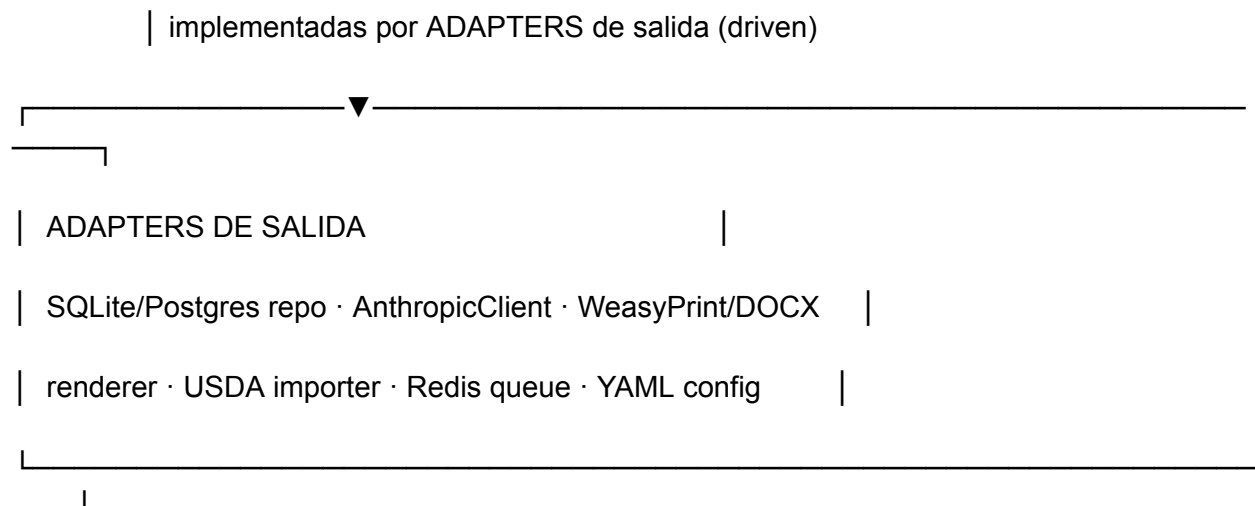
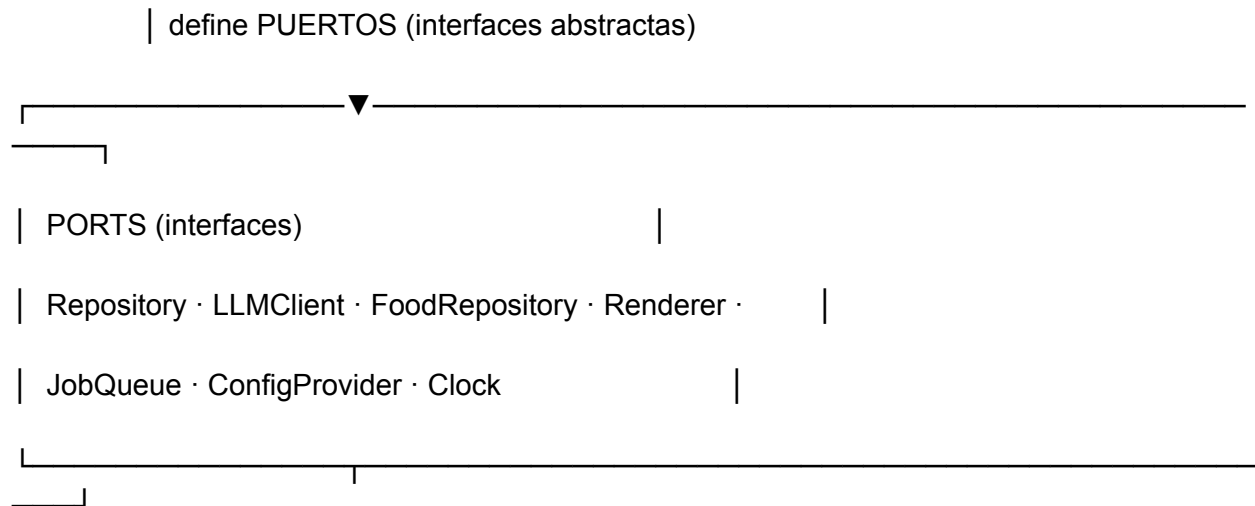
2. Principios de diseño (los mandamientos)

1. **Determinismo en los números.** Todo cálculo de kcal, macros y gramos ocurre en código puro, sin IA, reproducible bit a bit. La IA jamás produce una cifra que llegue al plan final.
 2. **La IA propone, el código dispone.** La IA elige *qué* alimentos y aporta variedad/redacción; el código calcula *cuánto* y valida contra los objetivos.
 3. **Núcleo puro, bordes sucios.** El dominio (cálculo, generación, reglas) no conoce Streamlit, ni FastAPI, ni SQLite, ni la API de Claude, ni WeasyPrint. Todo eso son adaptadores detrás de *puertos* (interfaces). Arquitectura hexagonal / ports & adapters.
 4. **Multi-tenant desde la primera línea.** Cada fila de datos lleva `tenant_id` aunque al principio haya un solo tenant. Migrar a multi-usuario es entonces trivial.
 5. **Contrato primero.** Los `schemas` (Pydantic) son la fuente de verdad de cada frontera. Se validan en cada borde: nada entra al dominio sin validar.
 6. **Reproducibilidad y auditoría.** Cada plan guarda su *procedencia*: versión de config, versión de prompt, modelo usado, hash de la entrada. Dado lo mismo, se puede regenerar idéntico.
 7. **Humano en el bucle.** El plan generado es un *borrador* que el profesional revisa, ajusta y aprueba. El sistema nunca entrega algo directamente al cliente final sin esa compuerta.
 8. **Fallar claro, no callado.** Errores tipados y explícitos, nunca `except: pass`. Un intake ambiguo se *marca para revisión*, no se adivina.
 9. **Config sobre código.** Estrategia nutricional (factores, deltas, %grasa, reparto por comida, tolerancias) vive en archivos de configuración versionados, no incrustada en Python.
 10. **Todo lo pesado es un job.** Generar y renderizar se modelan como trabajos con estado e idempotencia, listos para moverse a una cola sin cambiar la lógica.
-

3. Arquitectura general

3.1 Capas (de adentro hacia afuera)





La clave: el dominio y la aplicación dependen de *puertos* (interfaces), no de implementaciones. Cambiar SQLite→Postgres, Streamlit→FastAPI, Claude→Ollama = cambiar un adaptador, inyectarlo distinto. Cero cambios en la lógica.

3.2 Inyección de dependencias

Un *composition root* (un solo lugar, p. ej. `app/container.py`) arma el sistema: decide qué adaptador concreto se inyecta según el entorno (`ENV=local` → SQLite + Streamlit-in-process; `ENV=prod` → Postgres + Redis + FastAPI). Nada más en el código sabe qué implementación se usa.

4. Stack tecnológico

Capa	Tecnología	Por qué
Lenguaje	Python 3.12+	Ecosistema de datos/IA, tipado moderno
Validación / schemas	Pydantic v2	Contratos en cada borde; integra con Structured Outputs de Claude
Núcleo de dominio	Python puro + dataclasses /Pydantic	Sin dependencias de I/O
ORM / persistencia	SQLAlchemy 2.x (async)	Abstrae SQLite↔Postgres; repository pattern
Migraciones	Alembic	Esquema versionado y reproducible
API	FastAPI	Contract-first, async, OpenAPI gratis (se agrega en Nivel 2)
UI Nivel 1	Streamlit	Interfaz cómoda sin backend web
IA	Anthropic SDK (anthropic) + Structured Outputs	JSON garantizado por esquema
Optimización de porciones	numpy (v1) / scipy / pulp (v2 opcional)	Solver determinista
Datos de alimentos	pandas + SQLite/Postgres	Curaduría e ingest de USDA
Render PDF	Jinja2 + WeasyPrint	HTML/CSS → PDF, controlable y con marca
Render DOCX	python-docx	Salida editable opcional
Lectura de Word	python-docx + mammoth (texto)	Extracción del intake

Capa	Tecnología	Por qué
Config	YAML + Pydantic Settings	Estrategia ajustable, versionada
Cola de trabajos (Nivel 3)	Arq o Celery + Redis	Generación/render asíncronos
Logging	<code>structlog</code> (JSON)	Trazabilidad con <code>correlation_id</code>
Tests	<code>pytest</code> , <code>hypothesis</code> , <code>syrupy</code>	Unit, property-based, snapshot
Gestión de deps	<code>uv</code> o Poetry	Reproducible
Contenedor	Docker	Paridad local/prod

Modelos de Claude (verifica la lista vigente en <https://docs.claude.com/en/api/overview> antes de fijar):

- Ingesta (Word→JSON): un modelo económico y rápido (p. ej. Haiku) — tarea estructurada y acotada.
- Generación (selección de alimentos + variedad): un modelo de mayor calidad (p. ej. Sonnet).
- Ambas tareas usan **Structured Outputs** (`output_config.format` con JSON Schema, o pasando el modelo Pydantic directamente en el SDK de Python). Esto elimina el parseo frágil y garantiza el esquema.

5. Estructura del proyecto

nutriplan/

└─ pyproject.toml

└─ README.md

└─ .env.example

└─ docker-compose.yml

- | — config/
- | | — nutrition.default.yaml # estrategia nutricional (versionada)
- | | — branding/ # logos/colores por tenant
- | — prompts/
- | | — intake_extraction.v1.md # prompt de ingesta (versionado)
- | | — plan_generation.v1.md # prompt de generación (versionado)
- | — src/nutriplan/
- | | — domain/ # NÚCLEO PURO — sin I/O
- | | | — models.py # entidades y value objects
- | | | — calculation.py # Mifflin, TDEE, macros, reparto
- | | | — portioning.py # portion solver (gramos)
- | | | — generation_rules.py # estructura de comidas, variedad
- | | | — validation.py # tolerancias, invariantes
- | | | — errors.py # taxonomía de errores
- | | — ports/ # INTERFACES (abstract)
- | | | — repository.py
- | | | — food_repository.py
- | | | — llm_client.py
- | | | — renderer.py
- | | | — job_queue.py
- | | | — config_provider.py
- | — application/ # CASOS DE USO (orquestran dominio+puertos)

- | | | — parse_intake.py
- | | | — compute_targets.py
- | | | — generate_plan.py
- | | | — approve_plan.py
- | | | — export_plan.py
- | | — adapters/ # IMPLEMENTACIONES concretas
- | | | — db/ # SQLAlchemy: modelos, repos, session
- | | | | — models.py
- | | | | — repositories.py
- | | | | — session.py
- | | | — llm/
- | | | | — anthropic_client.py
- | | | | — ollama_client.py # opcional (100% local)
- | | | — food/
- | | | | — usda_importer.py
- | | | — render/
- | | | | — pdf_weasyprint.py
- | | | | — docx_renderer.py
- | | | — queue/
- | | | — arq_queue.py
- | | — api/ # FastAPI (Nivel 2)

```
| | └─ main.py
| | └─ routes/
| | └─ deps.py          # auth, tenant context
| | └─ schemas.py       # DTOs de request/response
| └─ ui/                # Streamlit (Nivel 1)
| | └─ app.py
| └─ config/
| | └─ settings.py      # Pydantic Settings (.env)
| └─ observability/
| | └─ logging.py
| └─ container.py       # composition root (DI)
└─ migrations/         # Alembic
└─ data/
| └─ foods/            # CSV curado + SQLite de alimentos
└─ tests/
    └─ unit/
    └─ property/
    └─ integration/
    └─ fixtures/       # Words de ejemplo, respuestas LLM grabadas
```

6. Modelo de dominio

Entidades y *value objects* principales (Pydantic v2). Todos los IDs son UUID. Todos los agregados con datos de personas llevan `tenant_id`.

```
# domain/models.py (esquemático)
```

```
from enum import Enum
```

```
from datetime import date, datetime
```

```
from pydantic import BaseModel, Field
```

```
from uuid import UUID
```

```
class Sex(str, Enum): FEMALE="female"; MALE="male"
```

```
class Goal(str, Enum): LOSE_FAT="lose_fat"; MAINTAIN="maintain";  
GAIN_MUSCLE="gain_muscle"
```

```
class ActivityLevel(str, Enum):
```

```
    SEDENTARY="sedentary"; LIGHT="light"; MODERATE="moderate"; ACTIVE="active";  
    VERY_ACTIVE="very_active"
```

```
class MealSlot(str, Enum):
```

```
    BREAKFAST="desayuno"; SNACK_AM="snack_am"; LUNCH="almuerzo";  
    SNACK_PM="snack_pm"; DINNER="cena"
```

```
class FoodCategory(str, Enum):
```

```
    PROTEIN="protein"; CARB="carb"; FAT="fat"; FRUIT="fruit"; VEGETABLE="vegetable";  
    DAIRY="dairy"; OTHER="other"
```

```
class Client(BaseModel):
```

```
    id: UUID
```

```
    tenant_id: UUID
```

```
    name: str
```

sex: Sex

birthdate: date | None = None # preferible a edad fija

age_years: int | None = None # fallback si no hay fecha

height_cm: float

weight_kg: float

goal: Goal

activity_level: ActivityLevel

liked_food_ids: list[UUID] = [] # alimentos que le gustan

restrictions: list[str] = [] # tags: "no_seafood", "no_gluten", "no_shake", ...

notes: str | None = None

class FoodItem(BaseModel):

id: UUID

tenant_id: UUID | None = None # None = alimento global compartido; con id = custom del tenant

source: str # "USDA" | "TCAC" | "custom"

source_ref: str | None = None # p.ej. fdc_id de USDA

name_es: str

name_en: str | None = None

category: FoodCategory

kcal_100g: float

protein_100g: float

carb_100g: float

fat_100g: float

tags: list[str] = [] # "mariscos", "gluten", "lacteo", "vegano", ...

default_unit_g: float | None = None # gramos de una porción típica (1 huevo≈50g)

class MacroTargets(BaseModel):

kcal: float

protein_g: float

carb_g: float

fat_g: float

class NutritionTargets(BaseModel):

id: UUID

tenant_id: UUID

client_id: UUID

daily: MacroTargets

per_meal: dict[MealSlot, MacroTargets]

method: str = "mifflin_st_jeor"

config_version: str # procedencia

overrides: dict = {} # ajustes manuales del entrenador

computed_at: datetime

class MealFoodPortion(BaseModel):

food_id: UUID

grams: float

```

class MealEntry(BaseModel):

    slot: MealSlot

    portions: list[MealFoodPortion]

    computed: MacroTargets          # recalculado por el código, no por la IA

    free_salad: bool = False

    free_protein: bool = False      # "proteína libre" del formato

class DayPlan(BaseModel):

    day_index: int                  # 0..6 (lunes..domingo)

    meals: list[MealEntry]

    totals: MacroTargets

class PlanPhase(str, Enum): FIRST_15="first_15"; NEXT_15="next_15"

class PlanStatus(str, Enum): DRAFT="draft"; APPROVED="approved"; EXPORTED="exported"

class PlanCycle(BaseModel):

    id: UUID

    tenant_id: UUID

    client_id: UUID

    targets_id: UUID

    phase: PlanPhase

    days: list[DayPlan]            # 7 días

    status: PlanStatus = PlanStatus.DRAFT

    # Procedencia (reproducibilidad):

    config_version: str

```

prompt_version: str

model: str

input_hash: str

created_by: UUID | None = None

created_at: datetime

approved_at: datetime | None = None

```
class IntakeStatus(str, Enum): PARSED="parsed"; NEEDS_REVIEW="needs_review";  
CONFIRMED="confirmed"
```

```
class IntakeDocument(BaseModel):
```

id: UUID

tenant_id: UUID

client_id: UUID | None

source_filename: str

raw_text: str

parsed: dict # datos extraídos por la IA

ambiguities: list[str] = [] # campos dudosos para revisión humana

status: IntakeStatus

created_at: datetime

Invariantes de dominio (se prueban):

- La suma de **per_meal** reconstruye **daily** dentro de ± 1 g por redondeo.
- Toda **MealEntry.computed** se calcula a partir de **portions** y la base de alimentos; nunca se confía en un número provisto por la IA.
- Todo **food_id** usado en un plan pertenece al conjunto de alimentos permitidos del cliente (preferencias $\cap$ no-restringidos).

7. Persistencia y multi-tenancy

7.1 Regla de oro

Cada tabla con datos de cliente lleva `tenant_id UUID NOT NULL` indexado. Los alimentos globales (USDA) viven en una tabla `foods` con `tenant_id NULL`; los alimentos custom de un tenant llevan su `tenant_id`.

7.2 Repository pattern con filtro automático de tenant

Toda consulta pasa por un repositorio que **inyecta el filtro `tenant_id` desde el contexto de la petición**. Nunca se escribe SQL crudo sin ese filtro. Esto hace imposible, por construcción, que un tenant vea datos de otro.

ports/repository.py (interfaz)

```
class ClientRepository(Protocol):
```

```
    async def add(self, client: Client) -> None: ...
```

```
    async def get(self, client_id: UUID) -> Client | None: ...
```

```
    async def list(self) -> list[Client]: ...
```

la implementación SQLAlchemy recibe el `tenant_id` del contexto y lo aplica siempre

7.3 SQLite → Postgres

- SQLAlchemy con URL configurable: `sqlite+aiosqlite:///./data/app.db` (local) o `postgresql+asyncpg:///...` (prod).
- Alembic para migraciones desde el día 1 (incluso en SQLite).
- En Postgres, activar **Row-Level Security** por `tenant_id` como segunda barrera (defensa en profundidad) en Nivel 3.

7.4 Tablas principales

`tenants`, `users` (coaches), `clients`, `foods`, `client_food_preferences`, `intake_documents`, `nutrition_targets`, `plan_cycles`, `day_plans`, `meal_entries`, `generation_jobs`, `export_artifacts`, `audit_log`.

8. Módulo 1 — Ingesta (Word → JSON estructurado)

8.1 Responsabilidad

Convertir un `.docx` de intake (texto libre y semiestructurado) en un objeto `IntakeParsed` validado, marcando ambigüedades para revisión humana.

8.2 Flujo

1. Leer el `.docx` (`mammoth/python-docx`) → `raw_text`.
2. Llamar al `LLMClient.extract(raw_text, schema=IntakeParsed)` usando **Structured Outputs** con el esquema Pydantic. El esquema garantiza forma y tipos.
3. Validar con Pydantic (segunda barrera).
4. **Detección de ambigüedad**: si un valor viene marcado como aproximado/dudoso (p. ej. "peso 71 kg aproximado, lo confirmo"), o falta un campo obligatorio, se agrega a `ambiguities` y el estado pasa a `NEEDS_REVIEW`. **La IA no rellena huecos por su cuenta.**
5. Persistir `IntakeDocument`.

8.3 Esquema de extracción (contrato de salida de la IA)

```
class IntakeParsed(BaseModel):
```

```
    model_config = ConfigDict(extra="forbid") # requerido por Structured Outputs
```

```
    name: str
```

```
    sex: Sex
```

```
    age_years: int | None
```

```
    height_cm: float | None
```

```
    weight_kg: float | None
```

```
    weight_is_approximate: bool = False      # bandera de ambigüedad
```

```
    city: str | None
```

```
    goal_raw: str                            # texto libre del objetivo
```

```
    liked_foods: LikedFoods                  # listas por categoría (strings)
```

```
restrictions_raw: list[str]          # "sin mariscos", etc.
```

```
uses_protein_shake: bool | None
```

```
training_raw: str | None
```

```
class LikedFoods(BaseModel):
```

```
    model_config = ConfigDict(extra="forbid")
```

```
    proteins: list[str] = []
```

```
    carbs: list[str] = []
```

```
    fats: list[str] = []
```

```
    fruits: list[str] = []
```

```
    vegetables: list[str] = []
```

8.4 Mapeo de alimentos (texto → **FoodItem**)

Los alimentos vienen como texto ("pollo", "arroz integral"). Un paso determinista los resuelve contra la base de alimentos por *matching* (exacto → normalizado → fuzzy con umbral). Los que no matchean se marcan como *no reconocidos* para que el entrenador los mapee o cree como alimento custom. Nunca se adivina un macro.

8.5 Guardrail

La IA **normaliza y estructura**, no interpreta clínicamente ni inventa datos faltantes. Si el objetivo o una restricción son ambiguos, se conservan en crudo y los resuelve el humano.

9. Módulo 2 — Motor de cálculo (determinista y configurable)

9.1 Responsabilidad

De **Client** + **config** → **NutritionTargets** (diario + reparto por comida). **Código puro, sin IA, 100% testeable.**

9.2 Fórmulas

- **BMR (Mifflin-St Jeor):**
 - Mujer: $10 * \text{peso_kg} + 6.25 * \text{altura_cm} - 5 * \text{edad} - 161$
 - Hombre: $10 * \text{peso_kg} + 6.25 * \text{altura_cm} - 5 * \text{edad} + 5$
- **TDEE** = $\text{BMR} * \text{factor_actividad}$
- **kcal objetivo** = $\text{TDEE} * (1 + \text{delta_objetivo})$
- **Proteína (g)** = $\text{proteina_g_por_kg} * \text{peso_kg}$
- **Grasa (g)** = $(\text{kcal_objetivo} * \text{pct_grasa}) / 9$
- **Carbohidratos (g)** = $(\text{kcal_objetivo} - \text{proteina_g} * 4 - \text{grasa_g} * 9) / 4$
- **Reparto por comida:** cada macro se multiplica por el % del slot en `meal_distribution`.

9.3 Config (fuente de verdad de la estrategia)

config/nutrition.default.yaml

version: "2026.07.01" # se guarda en cada plan (procedencia)

activity_factors:

sedentary: 1.20

light: 1.375

moderate: 1.55

active: 1.725

very_active: 1.90

goal_adjustments: # fracción sobre TDEE

lose_fat: -0.18

maintain: 0.0

gain_muscle: 0.10

protein_g_per_kg:

lose_fat: 2.0

maintain: 1.8

gain_muscle: 2.0

fat_pct_of_kcal: 0.28

meal_distribution: # debe sumar 1.0

desayuno: 0.25

snack_am: 0.10

almuerzo: 0.30

snack_pm: 0.10

cena: 0.25

tolerances: # desviación aceptada al validar

kcal: 0.08

protein_g: 0.10

carb_g: 0.12

fat_g: 0.12

portioning:

grams_rounding: 5 # redondear porciones a múltiplos de 5 g

min_portion_g: 20

9.4 Overrides por cliente

El entrenador puede fijar macros manualmente (lo que se vio en el mockup: campos editables). Si hay override, el motor **usa el override y registra overrides en NutritionTargets** para trazabilidad. El reparto por comida se recalcula sobre los macros finales.

9.5 Pruebas mínimas

- *Golden values*: casos con resultado conocido (calculado a mano) → asserts exactos con tolerancia de redondeo.

- Propiedad: `sum(per_meal) == daily` (± 1 g).
 - Propiedad: `protein_g*4 + carb_g*4 + fat_g*9 ≈ kcal` ($\pm 1\%$).
-

10. Módulo 3 — Base de alimentos (USDA)

10.1 Responsabilidad

Proveer una base **local, curada y etiquetada** de alimentos con macros por 100 g, consultable por categoría y restricciones.

10.2 Ingest

- Fuente primaria: **USDA FoodData Central** (descarga masiva o API, en inglés). Fuente secundaria opcional para alimentos locales: **TCAC (ICBF, Colombia)**.
- Un importador (`usda_importer.py`) toma una **lista curada** de ~60–150 alimentos (los que los entrenadores usan de verdad), extrae kcal/proteína/carbo/grasa por 100 g, normaliza nombres (ES/EN) y asigna `category + tags`.
- Resultado: se carga a la tabla `foods` (global, `tenant_id NULL`). No se consulta la API de USDA en caliente durante la generación.

10.3 Esquema y etiquetas

- Cada alimento: macros por 100 g + `tags` que habilitan filtros de restricción: `mariscos`, `gluten`, `lacteo`, `vegano`, `cerdo`, etc.
- `default_unit_g` para ítems contables (1 huevo $\approx$ 50 g, 1 banano $\approx$ 120 g) — útil para redactar el plan en unidades naturales además de gramos.

10.4 Filtro de restricciones (determinista)

Dado un cliente, se computa su **conjunto permitido**: `alimentos que le gustan` menos los que tengan cualquier `tag` prohibido por sus `restrictions`. Ese conjunto es el único universo que verá la IA en el Módulo 4.

10.5 Alimentos custom

Un tenant puede crear alimentos propios (`tenant_id` = su id). Se combinan con los globales al construir el conjunto permitido.

11. Módulo 4 — Generación del plan (IA propone, código dispone)

Este es el corazón. Es un bucle **propuesta** → **cálculo** → **validación** → **(reintento)**.

11.1 Estructura de comidas (reglas de dominio)

Cada slot tiene una "receta estructural" (no cantidades, solo roles), tomada del formato existente:

- **Desayuno:** base de huevos/proteína + 1 carbo + 1 grasa (o receta tipo pancakes de avena).
- **Snack AM/PM:** lácteo/proteína ligera + fruta.
- **Almuerzo:** proteína libre + carbo (≈120 g cocido) + ensalada libre (± aguacate).
- **Cena:** proteína + carbo menor (≈60 g) o solo proteína + ensalada.

Estas reglas viven en `generation_rules.py` como datos, no como texto.

11.2 Paso 1 — La IA elige alimentos (no gramos)

Para cada día y slot, la IA selecciona *qué* alimentos del **conjunto permitido** usar, respetando la estructura y aportando **variedad** (no repetir la misma proteína/carbo demasiados días).

Guardrail fuerte con Structured Outputs: el esquema de salida define `food_id` como un **enum limitado al conjunto permitido del cliente**. Así el modelo *no puede* elegir un alimento fuera de la lista ni inventarlo.

```
class MealSelection(BaseModel):
```

```
    model_config = ConfigDict(extra="forbid")
```

```
    slot: MealSlot
```

```
    food_ids: list[str]    # enum restringido en tiempo de ejecución al conjunto permitido
```

```
    free_salad: bool = False
```

```
class DaySelection(BaseModel):
```

```
    model_config = ConfigDict(extra="forbid")
```

```
    day_index: int
```

```
meals: list[MealSelection]
```

```
class PlanSelection(BaseModel):
```

```
    model_config = ConfigDict(extra="forbid")
```

```
    days: list[DaySelection] # 7 días
```

El esquema JSON se construye dinámicamente por cliente, inyectando el `enum` de `food_ids` permitidos.

11.3 Paso 2 — El código calcula los gramos (portion solver)

Dado el conjunto de alimentos elegidos para un slot y los macros objetivo del slot, el solver determinista calcula los gramos.

Estrategia v1 (roles): asignar cada alimento a su macro dominante (proteína/carbo/grasa según su categoría), y resolver los gramos para cubrir cada macro desde su fuente principal, ajustando por los aportes cruzados. Redondear a múltiplos de `grams_rounding`, respetar `min_portion_g`.

Estrategia v2 (optimización): programación lineal (`scipy.optimize/pulp`) que minimiza la desviación ponderada respecto a los macros objetivo sujeta a límites de porción. Más precisa; se activa por config.

11.4 Paso 3 — Validación

Recalcular los macros reales del slot y del día a partir de las porciones y la base de alimentos. Comparar contra objetivo con las `tolerances` de la config.

- Si **cuadra**: aceptar.
- Si **no cuadra** por porciones: el solver ya intentó ajustar; si aún no cuadra, marcar el slot y probar otra combinación (volver al Paso 1 para ese slot/día, hasta `max_retries`).
- Si tras `max_retries` no cuadra: lanzar `GenerationError` con detalle; el caso queda para revisión humana (nunca se entrega un plan fuera de tolerancia sin marca).

11.5 Variedad (regla determinista, no dejada a la IA)

Se lleva un registro de proteínas/carbos usados en la semana; se pasa a la IA como restricción ("no repitas más de N veces") y se verifica en código. La variedad también es medible como métrica de calidad.

11.6 Idempotencia

`input_hash = hash(client_snapshot + targets + config_version + prompt_version + allowed_food_set)`. Si ya existe un `PlanCycle` con ese hash, se devuelve (no se regenera). Esto ahorra costo de IA y hace la generación reproducible.

11.7 Salida

Dos `PlanCycle` (fase `first_15` y `next_15`), cada uno con 7 `DayPlan`, en estado `DRAFT`, con procedencia completa.

12. Módulo 5 — Renderizado (PDF / DOCX)

12.1 Responsabilidad

De un `PlanCycle` aprobado → artefacto PDF (o DOCX) con la marca del tenant, idéntico al formato del negocio.

12.2 Diseño

- Plantilla **Jinja2 HTML/CSS** que reproduce el formato (tabla semanal, 5 comidas, "Anotaciones Importantes" fijas) → **WeasyPrint** → PDF.
- Branding por tenant (logo, colores, handle) inyectado desde `config/branding/<tenant>`.
- Alternativa DOCX editable con `python-docx` (misma data, otro renderer detrás del mismo puerto `Renderer`).
- Determinista: mismo plan → mismo PDF (byte-estable salvo metadatos de fecha).

12.3 Puerto

```
class Renderer(Protocol):
```

```
    async def render(self, plan: PlanCycle, branding: Branding, fmt: Literal["pdf","docx"]) -> bytes:
```

```
    ...
```

12.4 Almacenamiento

El artefacto se guarda en `export_artifacts` (local: filesystem; prod: object storage tipo S3) referenciando el `plan_cycle_id`.

13. Capa de IA (abstracción, prompts, costo, guardrails)

13.1 Puerto `LLMClient`

`class LLMClient(Protocol):`

`async def extract(self, text: str, schema: type[BaseModel], *, model: str) -> BaseModel: ...`

`async def select_plan(self, prompt: str, schema: type[BaseModel], *, model: str) -> BaseModel: ...`

Implementaciones: `AnthropicClient` (producción) y `OllamaClient` (opcional, 100% local, menor calidad). El dominio no sabe cuál se usa.

13.2 Structured Outputs

Ambas llamadas usan Structured Outputs de la Claude API (esquema JSON / modelo Pydantic). Esto **garantiza JSON conforme al esquema** y elimina parseo frágil. Aun así, se valida con Pydantic al recibir (defensa en profundidad).

13.3 Prompts versionados

Los prompts viven en `/prompts/*.vN.md`. La versión usada se guarda en cada plan (`prompt_version`). Cambiar un prompt = nueva versión, nunca edición silenciosa. Esto preserva reproducibilidad y permite A/B.

13.4 Reintentos y fallback

- Reintentos con backoff exponencial ante errores transitorios del proveedor.
- Si Structured Outputs falla el esquema tras N intentos → `LLMError` tipado; el caso va a revisión, no se entrega basura.

13.5 Costo

- **Prompt caching** para la parte estática (reglas, estructura de comidas, lista de alimentos permitidos) → menos tokens por llamada.
- **Batch API** para generación masiva (muchos clientes de un tenant de una vez) — asíncrono y más barato.
- Selección de modelo por tarea (Haiku ingesta / Sonnet generación) por config.
- Cada `GenerationJob` registra tokens y costo estimado (observabilidad de negocio).

13.6 Guardrails de seguridad (críticos)

- La IA **nunca** produce números finales; el código recalcula todo.
 - El `enum` de `food_ids` impide alimentos fuera de la lista permitida.
 - La IA no da consejo médico ni promete resultados; el plan es un borrador que el profesional firma.
 - Los datos del cliente enviados a la IA se minimizan a lo necesario para la tarea.
-

14. Orquestación y trabajos (jobs)

14.1 Casos de uso (application)

Cada caso de uso es una función delgada que orquesta dominio + puertos, sin lógica de negocio propia: `ParseIntake`, `ComputeTargets`, `GeneratePlan`, `ApprovePlan`, `ExportPlan`.

14.2 Modelo de job

`GeneratePlan` y `ExportPlan` se modelan como `GenerationJob/ExportJob` con estados `queued` → `running` → `done|failed`, `idempotency_key`, `input_hash`, `result_id`, `error`.

- **Nivel 1:** el job corre en proceso (síncrono o en un hilo), pero *ya se persiste como job con estado*. La UI muestra progreso leyendo el estado.
- **Nivel 3:** el mismo job se encola en Redis (Arq/Celery) y lo procesan *workers* sin estado. **La lógica no cambia:** solo el adaptador de cola.

14.3 Idempotencia

Un `idempotency_key` por petición evita generar dos veces lo mismo ante reintentos de red.

15. API (contract-first, se agrega en Nivel 2)

Aunque el Nivel 1 use Streamlit llamando los casos de uso en proceso, **se diseña la API desde ya** para que Streamlit y FastAPI sean dos adaptadores del mismo núcleo.

Endpoints REST (todos *tenant-scoped* vía auth):

POST /v1/clients crear cliente

GET /v1/clients/{id}

POST /v1/intake subir .docx → parsear (devuelve ambigüedades)

POST /v1/intake/{id}/confirm confirmar/corregir campos ambiguos

POST /v1/clients/{id}/targets calcular targets (o aplicar overrides)

POST /v1/clients/{id}/plans:generate encola job de generación → {job_id}

GET /v1/jobs/{id} estado del job

GET /v1/plans/{id} ver plan (draft/approved)

PATCH /v1/plans/{id} editar plan (ajuste manual del entrenador)

POST /v1/plans/{id}:approve aprobar (draft → approved)

POST /v1/plans/{id}:export?format=pdf render → {artifact_url}

GET /v1/foods?query=&category= buscar alimentos

POST /v1/foods crear alimento custom del tenant

- **Versionado** de API (/v1) desde el inicio.
- **OpenAPI** autogenerado por FastAPI.
- **Auth**: Nivel 1 sin auth (un tenant). Nivel 2: API key o JWT que incluye `tenant_id`; un *dependency* de FastAPI (`deps.py`) resuelve el contexto de tenant y lo inyecta a los repositorios.
- **DTOs** (`api/schemas.py`) separados de las entidades de dominio (no exponer el modelo interno directamente).

16. Sistema de configuración

- `config/nutrition.default.yaml` versionado (sección 9.3). Un tenant puede tener overrides (`nutrition.<tenant>.yaml`) que se fusionan sobre el default.
- Ajustes de entorno vía **Pydantic Settings** + `.env` (DB URL, claves de API, entorno, modelos por tarea).
- La `config_version` efectiva se guarda en cada plan → reproducibilidad.

Variables de entorno (`.env.example`):

ENV=local # local | prod

DATABASE_URL=sqlite+aiosqlite:///./data/app.db

ANTHROPIC_API_KEY=...

LLM_MODEL_INGEST=<modelo economico>

LLM_MODEL_GENERATE=<modelo calidad>

REDIS_URL= # vacío en local

OBJECT_STORAGE_URL= # vacío en local (usa filesystem)

LOG_LEVEL=INFO

17. Manejo de errores (taxonomía)

Errores de dominio tipados en `domain/errors.py`, mapeados a HTTP en la capa API:

Error	Cuándo	HTTP
<code>IntakeAmbiguityError</code>	Falta o es dudoso un dato del intake	422 (needs review)
<code>FoodNotFoundError</code>	Alimento de la lista no está en la base	422
<code>CalculationError</code>	Datos insuficientes para calcular targets	422
<code>GenerationError</code>	No se logró cuadrar macros tras <code>max_retries</code>	422
<code>RenderError</code>	Falla al generar PDF/DOCX	500
<code>LLMError</code>	Falla del proveedor / esquema inválido	502

Error	Cuándo	HTTP
<code>TenantIsolationError</code>	Acceso cruzado entre tenants (nunca debería pasar)	403

Regla: **nada de `except: pass`**. Todo error se registra con contexto y, si es de datos, se convierte en "marcar para revisión", no en una suposición.

18. Observabilidad

- **Logging estructurado** (`structlog`, JSON) con `correlation_id`, `tenant_id`, `client_id`, `job_id` en cada línea. Sin PII en logs (nombres/medidas se referencian por id).
 - **Métricas** de negocio y técnicas: latencia de generación, tokens y costo por plan, tasa de "cuadra a la primera", número de reintentos, planes que van a revisión.
 - **Procedencia** en cada plan (`config_version`, `prompt_version`, `model`, `input_hash`) = auditoría completa: siempre se puede responder "¿con qué reglas y qué modelo se generó este plan?".
 - **audit_log**: quién aprobó/editó/exportó qué y cuándo.
-

19. Estrategia de pruebas

Tipo	Qué cubre
Unit	Motor de cálculo (golden values), portion solver, filtros de restricción, mapeo de alimentos
Property-based (Hypothesis)	Para <i>cualquier</i> cliente válido: (a) macros del plan dentro de tolerancia; (b) todos los <code>food_id</code> $\in$ conjunto permitido; (c) <code>sum(per_meal)=daily</code>
Snapshot (syropy)	Render de PDF/DOCX estable ante cambios no intencionados

Tipo	Qué cubre
Contract / LLM	Proveedor de IA <i>mockeado</i> con respuestas grabadas (fixtures) — determinista, sin llamadas reales en CI
Integración	Pipeline completo sobre los Words de ejemplo reales → plan válido
Aislamiento de tenant	Un tenant nunca puede leer datos de otro (test explícito)

Cobertura obligatoria: 100% del **domain/** (es puro y crítico). El resto, razonable.

20. Seguridad y privacidad

- **Datos de salud** (peso, medidas, fotos si se agregan): categoría sensible. En Colombia aplica la **Ley 1581 (Habeas Data)**: consentimiento, finalidad, minimización, derecho de supresión.
 - **Aislamiento por tenant**: filtro obligatorio + (en Postgres) Row-Level Security.
 - **Secretos** fuera del código (**.env** no versionado; en prod, secret manager).
 - **Cifrado en reposo** de la base en Nivel 3; TLS en tránsito siempre en prod.
 - **Sin PII en logs**; retención de datos configurable; borrado por cliente.
 - **Encuadre profesional**: el sistema genera un *borrador* que el entrenador (profesional) revisa y firma; sin afirmaciones médicas ni promesas de resultado. Esto es correcto y limita responsabilidad.
-

21. Escalabilidad — el camino sin re-arquitectura

Lo que **cambia** por nivel es el *adaptador*; lo que **no cambia** es el **domain/** + **application/**.

Nivel	UI / Entrada	DB	Jobs	IA	Tenants
1 — Local	Streamlit en proceso	SQLite	síncrono (pero persistido como job)	Anthropic directo	1 (con tenant_id ya presente)

Nivel	UI / Entrada	DB	Jobs	IA	Tenants
2 — API multi-coach	FastAPI + Streamlit	Postgres	síncrono/hilo	+ prompt caching	N (auth + tenant context)
3 — SaaS escalable	FastAPI tras balanceador, réplicas <i>stateless</i>	Postgres + réplicas de lectura	Redis + workers	+ Batch API, caching	N, con cobros (Stripe)
4 — Escala grande	Autoscaling, CDN de assets	Partición por tenant, multi-región	Autoscaling de workers	Colas dedicadas	Miles

Palancas de escalado horizontal ya previstas por el diseño:

- Servicios *stateless* → se replican detrás de un balanceador sin sesión pegajosa.
- Trabajo pesado en cola → se agregan workers para más throughput.
- Datos particionados por `tenant_id` → la base se parte por tenant cuando haga falta.
- Caching de la base de alimentos y de prompts → menos I/O y menos costo de IA.
- Object storage para PDFs → no se saturan discos de aplicación.

Por eso "robusto en Nivel 1" y "escalable" no compiten: se logra lo mismo escribiendo el núcleo limpio una sola vez.

22. Roadmap de construcción (orden para Claude Code)

Construir **en este orden**, cada paso con sus tests antes de seguir:

1. **Andamiaje**: estructura de carpetas, `pyproject`, settings, logging, `container.py`, SQLAlchemy base + Alembic init (SQLite).
2. **Dominio: modelos** (`domain/models.py`) + errores. Tests de invariantes.
3. **Módulo 2 — Motor de cálculo** + `config/nutrition.default.yaml`. Tests golden + property. *(Es el cimiento determinista y no depende de nada externo.)*
4. **Módulo 3 — Base de alimentos**: importador USDA + CSV curado + tabla `foods` + filtro de restricciones. Tests.
5. **Puertos + adaptador DB** (repositorios con filtro de tenant) + migraciones.
6. **Módulo 5 — Render** (Jinja2 + WeasyPrint) contra un plan de prueba fijo. Snapshot test. *(Se hace antes de la IA para tener salida visible temprano.)*

7. **Capa de IA:** puerto `LLMClient` + `AnthropicClient` con Structured Outputs + prompts v1. Proveedor mock para tests.
8. **Módulo 1 — Ingesta** (Word→JSON) usando la capa de IA. Tests con los Words de ejemplo.
9. **Módulo 4 — Generación:** reglas de estructura, selección por IA (enum de food_ids), portion solver v1, validación, variedad, idempotencia. Tests property (todo plan dentro de tolerancia).
10. **Casos de uso** (application) que unen todo: `ParseIntake` → `ComputeTargets` → `GeneratePlan` → `ApprovePlan` → `ExportPlan`.
11. **UI Streamlit** (Nivel 1) sobre los casos de uso: la pantalla del mockup (cliente, macros editables, chips de alimentos, restricciones, generar, vista previa editable, exportar).
12. **Nivel 2 (cuando aplique):** FastAPI + auth + tenant context + Postgres. El núcleo no se toca.

Entregable mínimo viable (fin del paso 11): Valeria sube un Word, revisa/ajusta, genera el plan de 30 días y exporta el PDF con su marca — todo local.

23. Requisitos no funcionales

- **Rendimiento:** generar un ciclo de 15 días en < 20 s (con IA) en Nivel 1; render PDF < 3 s.
 - **Reproducibilidad:** dado el mismo `input_hash`, mismo plan.
 - **Disponibilidad** (Nivel 3): objetivo 99.5% mensual.
 - **Costo por plan:** presupuestar tokens; usar caching y Batch para mantenerlo por debajo de un umbral definido por plan.
 - **Portabilidad:** corre en local (SQLite) y en nube (Postgres) sin cambios de código, solo config.
-

24. Anexos

24.1 Decisiones de arquitectura (resumen tipo ADR)

- **ADR-01:** Arquitectura hexagonal → permite escalar sin reescribir el núcleo. *Aceptada.*
- **ADR-02:** `tenant_id` desde el día 1 aunque haya un solo tenant. *Aceptada.*
- **ADR-03:** Números solo en código; IA solo selección/lenguaje. *Aceptada.*
- **ADR-04:** `food_id` como enum en Structured Outputs para impedir alucinaciones. *Aceptada.*
- **ADR-05:** Prompts y config versionados y guardados en cada plan. *Aceptada.*

- **ADR-06:** Trabajo pesado modelado como job desde Nivel 1. *Aceptada.*

24.2 Glosario

- **Tenant:** un entrenador (o cuenta) con sus clientes y datos aislados.
- **Conjunto permitido:** alimentos que le gustan al cliente menos los prohibidos por sus restricciones.
- **Procedencia:** metadatos que permiten reproducir/auditar un plan (config, prompt, modelo, hash).
- **Portion solver:** componente determinista que calcula gramos para cuadrar macros.
- **Structured Outputs:** capacidad de la Claude API que fuerza el esquema JSON de la respuesta.

24.3 Referencias

- Structured Outputs (Claude API):
<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- Prompt caching y Batch API: <https://docs.claude.com/en/api/overview>
- USDA FoodData Central: <https://fdc.nal.usda.gov/>
- Ley 1581 de 2012 (Colombia, protección de datos personales).

Fin de la especificación. Construir por módulos, en orden, con pruebas verdes en cada paso. El núcleo se escribe una vez; los adaptadores crecen con el negocio.